Indian Institute of Technology Jodhpur

Department of Computer Science & Engineering



GPU Programming
(CSL7520)

ASSIGNMENT

Implementation and Performance Evaluation of GPU Accelerated SHA-256 Hashing

Submitted By-
Ankesh Kumar (g25ait1022)
Anudeep S (g25ait1026)
Nishtha Parashar (g25ait1213)
Vijender Singh Thakur (g25ait1188)

Submitted To-
Dr. Binod Kumar
Assistant Professor
IIT Jodhpur

Trimester-3, Cohort-1
Postgraduate Diploma in Artificial Intelligence

1. Objective

The objective of this assignment is to implement the SHA-256 cryptographic hashing algorithm using CUDA to exploit **GPU parallelism** for efficient hash computation. The implementation aims to evaluate the performance of GPU-based hashing by measuring runtime, throughput, scalability, and speedup across different NVIDIA GPU architectures **(GTX1650, MX250, and RTX4060)** for varying batch sizes. The results are further compared with CPU execution to demonstrate the advantages of GPU acceleration for large-scale cryptographic workloads.

2. SHA-256 Algorithm Overview

SHA-256 (Secure Hash Algorithm 256-bit) is a member of the SHA-2 family standardized by NIST in FIPS 180-4. It produces a 256-bit (32-byte) digest from an arbitrary-length input.

2.1 Algorithm Structure

SHA-256 processes input in 512-bit (64-byte) blocks through a Merkle–Damgard construction:

1. Message Padding — append 0x80, zero bytes, and 64-bit big-endian message length to make the total length a multiple of 512 bits.
2. Block Splitting — divide padded message into 512-bit blocks.
3. Message Schedule Expansion — expand each 16-word block into 64 32-bit words (W[0..63]) using mixing functions σ_0 and σ_1.
4. Compression Function — 64 rounds operating on eight 32-bit state words (a, b, c, d, e, f, g, h) using rotation and XOR operations.
5. Hash Accumulation — add compressed output to running hash state after each block.
6. Output — concatenate final eight state words as 256-bit digest.

2.2 Core Operations

Every round uses these bitwise primitives — all computable in a single GPU clock cycle:

Table 1. Core bitwise operations and logical functions used in the SHA-256 compression function

Operation	Description
ROTR	Bit rotation
CH	Conditional selection
MAJ	Majority function
Σ_0	State mixing
Σ_1	State mixing

σ_0	Message expansion
σ_1	Message expansion

2.3 Why SHA-256 is GPU-Friendly

Three properties make SHA-256 an ideal GPU workload:

- No data dependencies across messages — thread N never reads thread M's data
- Fixed, predictable memory access pattern — no dynamic pointer chasing or branch divergence between messages
- High arithmetic intensity — 64 rounds of computation per 64-byte input block; keeps GPU ALUs busy relative to memory bandwidth consumed.

3. Experimental Setup

- **Input:** "The quick brown fox jumps over the lazy dog"
- **Batch Size:** 1K, 10K, 100K, and 1M hashes
- **Algorithm:** SHA-256
- **Implementation:** CUDA C++
- **Platforms Tested:**
  - NVIDIA GeForce RTX 4060 Laptop GPU
  - NVIDIA GeForce GTX 1650
  - NVIDIA GeForce MX250

4. Implementation Highlights

The experimental evaluation demonstrates that:

- GPU acceleration substantially improves the computational efficiency of SHA-256.
- GPU performance scales with increasing batch size, whereas CPU throughput remains relatively constant.
- Higher-end GPU architectures provide greater throughput and lower execution time.
- The performance advantage of GPUs becomes increasingly significant for large workloads due to improved utilization of parallel processing resources.

5. Results & Discussion

To evaluate the effectiveness of the proposed CUDA-based SHA-256 implementation, performance experiments were conducted on **three** NVIDIA GPU platforms (GTX1650, MX250, and RTX4060). For each hardware configuration, **100** benchmark runs were performed using **four batch sizes** (1K, 10K, 100K, and 1M). Runtime and throughput were recorded, and statistical measures including mean and standard deviation were computed to ensure reliable performance evaluation. The summarized benchmark statistics are presented in Table 2, while graphical comparisons are provided in Figures 1–5.

Table 2. Performance Statistics of SHA-256 Implementation on CPU and GPU Platforms

Batch Size	Device	Processor	Runs	Runtime Mean (ms)	Runtime Std (ms)	Throughput Mean (MH/s)	Throughput Std (MH/s)
1000	CPU	GTX1650	100	0.32678	0.10337	3.19012	0.43344
10000	CPU	GTX1650	100	3.12839	0.22554	3.20959	0.19061
100000	CPU	GTX1650	100	31.18071	0.83161	3.20876	0.08256
1000000	CPU	GTX1650	100	315.82776	7.15103	3.16733	0.07044
1000	CPU	MX250	100	0.37923	0.06167	2.70282	0.41685
10000	CPU	MX250	100	3.46839	0.56684	2.93930	0.35346
100000	CPU	MX250	100	33.84672	3.25327	2.97526	0.22459
1000000	CPU	MX250	100	330.57816	4.42793	3.02499	0.04036
1000	CPU	RTX4060	100	0.21269	0.05110	4.80728	0.48329
10000	CPU	RTX4060	100	2.06711	0.07071	4.84241	0.15543
100000	CPU	RTX4060	100	21.53797	0.79682	4.64815	0.15552
1000000	CPU	RTX4060	100	208.54044	2.77807	4.79563	0.06265
1000	GPU	GTX1650	100	2.10988	1.64548	0.65781	0.28502
10000	GPU	GTX1650	100	2.54654	3.07222	9.46945	6.09195
100000	GPU	GTX1650	100	4.25330	6.55516	39.61738	11.40913
1000000	GPU	GTX1650	100	15.76887	17.12666	70.83303	7.85484
1000	GPU	MX250	100	1.23926	0.85587	0.87113	0.10943
10000	GPU	MX250	100	2.60019	12.46881	9.61428	3.42116
100000	GPU	MX250	100	4.50239	3.05535	24.89074	4.09196
1000000	GPU	MX250	100	33.99991	20.27850	31.08993	2.69144
1000	GPU	RTX4060	100	1.31747	1.53885	1.07608	0.39765
10000	GPU	RTX4060	100	1.34110	0.69485	8.50271	2.33714
100000	GPU	RTX4060	100	1.54595	0.31774	66.45490	8.06150
1000000	GPU	RTX4060	100	6.93533	0.81668	146.16500	16.66534

Table 2 summarizes the average runtime and throughput obtained from 100 benchmark runs for each CPU and GPU configuration. The results indicate that GPU implementations achieve substantially higher throughput and lower execution time than CPU implementations, with the RTX4060 delivering the best overall performance.

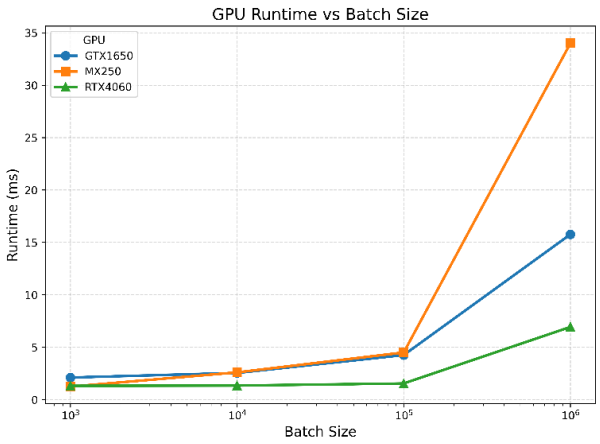


Figure 1. GPU Runtime vs Batch Size

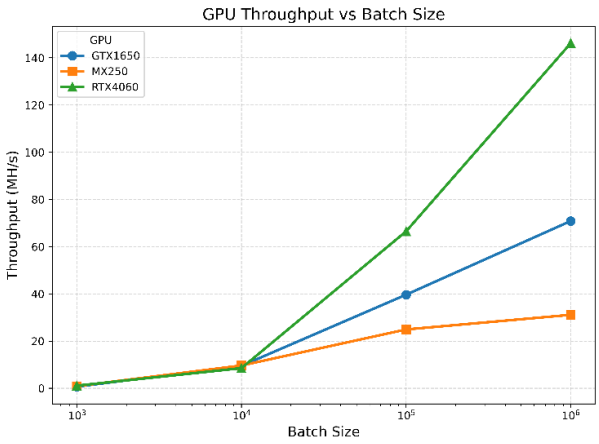


Figure 2. GPU Throughput vs Batch size

Figure 1 compares the runtime of the three GPU platforms for different batch sizes. The RTX4060 consistently achieves the lowest execution time, while the performance gap between the GPUs becomes more pronounced as the workload increases. **Figure 2** illustrates the throughput achieved by each GPU. Throughput increases with batch size for all platforms, with the RTX4060 achieving the highest throughput across all workloads.

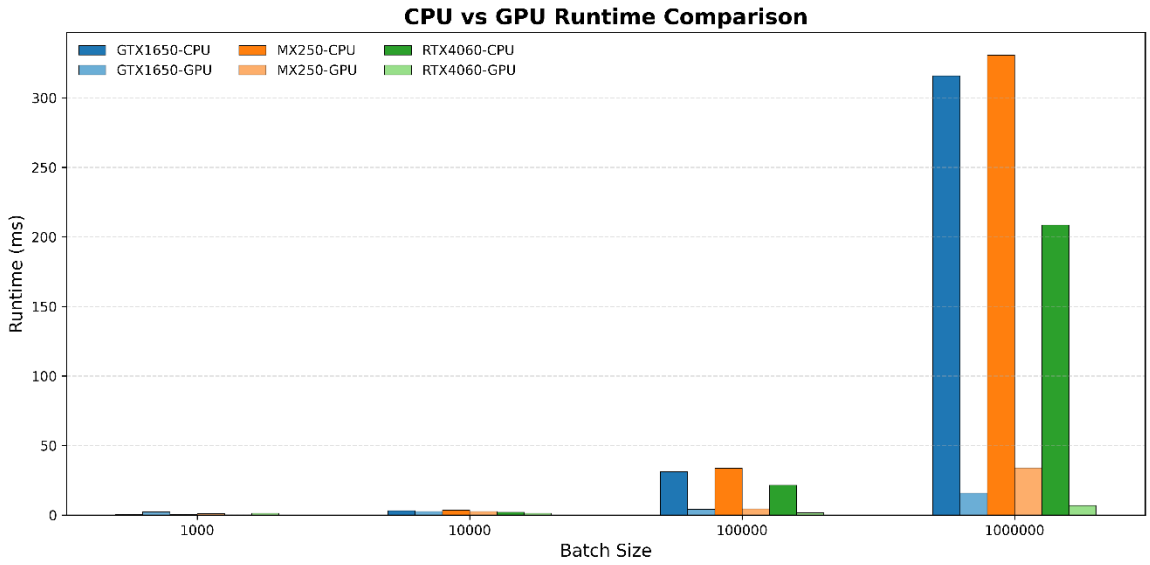


Figure 3. CPU vs GPU Runtime Comparison

Figure 3 presents the runtime comparison between CPU and GPU implementations. GPU execution provides significantly lower runtime than CPU execution, particularly for larger batch sizes where parallel processing is fully utilized.

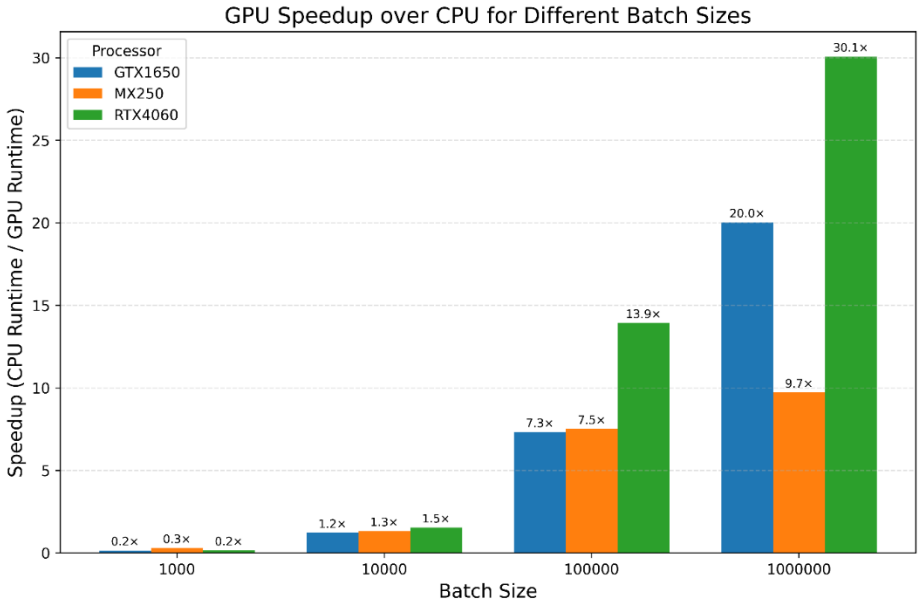


Figure 4. GPU Speedup over CPU for different batch sizes

Figure 4 shows the speedup achieved by GPU execution relative to CPU execution. Speedup increases with batch size, demonstrating the growing advantage of GPU parallelism for computationally intensive hashing tasks.

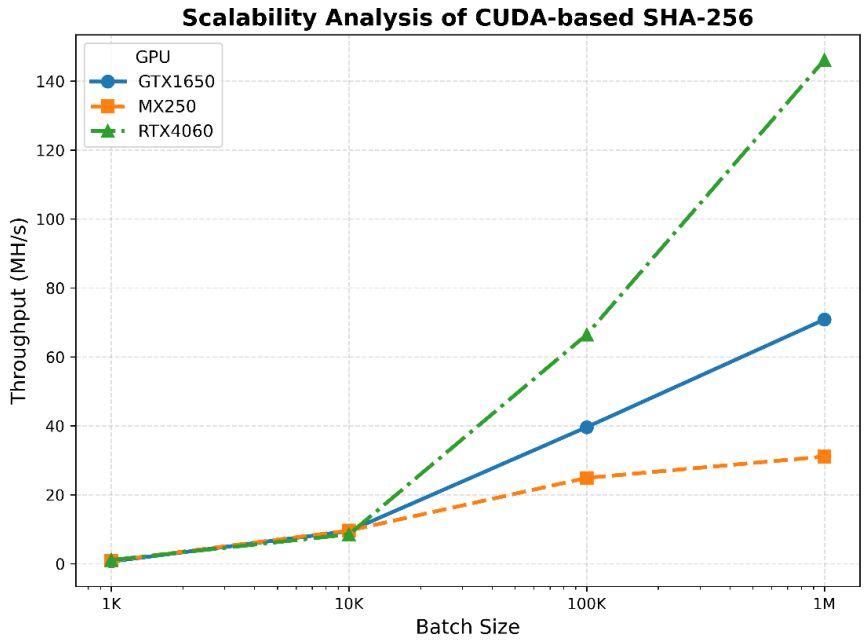


Figure 5. Scalability Analysis of CUDA-based SHA-256

Figure 5 illustrates the scalability of the GPU implementation. Throughput improves consistently with increasing workload, with the RTX4060 exhibiting the highest scalability, followed by the GTX1650 and MX250.

5. Conclusion

The experimental evaluation demonstrates the effectiveness of GPU acceleration for SHA-256 hashing. While CPU execution maintains relatively stable throughput across different batch sizes, GPU implementations benefit significantly from increased parallel workload. The **RTX4060** consistently outperformed the **GTX1650** and **MX250** in terms of runtime, throughput, scalability, and overall speedup. Experimental results indicate that GPU acceleration is particularly advantageous for large-scale cryptographic applications, where execution time can be reduced by up to **30**× compared with CPU execution. These findings validate the suitability of CUDA-enabled GPUs for high-performance hashing applications such as blockchain systems, password recovery, and large-scale data integrity verification.

6. References & Standards

- NIST FIPS 180-4, "Secure Hash Standard (SHS)," August 2015.
- RFC 6234, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," May 2011.
- NVIDIA CUDA C++ Programming Guide, v12.x. developer.nvidia.com/cuda-documentation
- NVIDIA PTX ISA Reference 8.x — byte_perm, rotation intrinsics.
- OpenCL 3.0 Specification, Khronos Group, 2020. khronos.org/opencl
- Bernstein & Lange, "Non-uniform caches," Fast Software Encryption 2007.
- NVIDIA Ampere Architecture Whitepaper, "NVIDIA A100 Tensor Core GPU Architecture," 2020.